BAHIR DAR UNIVERSITY

BAHIR DAR INSTITUTE OF TECHNOLOGY

FACULTY OF COMPUTING

DEPARTEMENT OF SOFTWARE ENGINEERING

Course title: Operating System and System Programming

Topic: The Installation, Evaluation, and Virtualization Analysis of HarmonyOS (OpenHarmony)

Name: Gideon Berhanu

ID Number: BDU1604303

Section: B

Submission Date: 20/08/2018 EC

Submitted To: Lec. Wendimu Baye

1. Installation of Operating System in Virtual Environment

a. Introduction

Background: HarmonyOS (HongmengOS) is a distributed operating system developed by Huawei. While research began in 2012, its development became a matter of corporate survival in May 2019 after U.S. trade restrictions severed Huawei's access to the Android ecosystem and Google Mobile Services (GMS). Initially launched for smart TVs (HarmonyOS 1.0), it has evolved through versions 2.0, 3.0, and 4.0, culminating in HarmonyOS NEXT. Unlike previous versions that utilized a Linux kernel with Android compatibility layers, HarmonyOS NEXT is a completely independent platform using a custom microkernel, representing Huawei's total technological independence.

Motivation: Huawei's motivation was twofold: Survival and Innovation. By moving away from U.S.-dependent technology, Huawei ensured its hardware could continue to function globally. Innovation-wise, they aimed to solve the "fragmentation" of modern technology. Instead of having separate OS architectures for phones, watches, and smart homes, HarmonyOS provides a unified platform. This allows multiple devices to act as a single "Super Device," where a task started on a phone can be finished on a tablet or TV seamlessly.

b. Objectives

The primary goal of this project is to document the technical hurdles and successful configurations involved in emulating an IoT-centric OS.

- **Deployment:** Install and configure DevEco Studio to manage virtual HarmonyOS devices.
- **Infrastructure Analysis:** Evaluate the hardware and software requirements necessary to sustain a microkernel-based emulation.
- **Problem-Solving:** Document specific errors (Missing images, Repo sync failures) and propose technical solutions.
- **Architectural Review:** Analyze the filesystem (EROFS, HMDFS) and the role of virtualization in modern deployment.

c. Requirements

i. Hardware Requirements

- **Processor:** A modern multi-core CPU is required. Recommended: Intel Core i5 (10th Gen+) or AMD Ryzen 5 3500U+. The CPU must support hardware virtualization (VT-x or AMD-V) enabled in the BIOS.
- **Memory (RAM):** A minimum of 8 GB is required for basic IDE operation, though 16 GB is strongly recommended for running the emulator alongside the development environment.
- **Storage:** At least 256 GB SSD. NVMe drives are preferred to handle the heavy read/write loads during OS compilation.
- **Graphics:** A GPU supporting OpenGL ES 3.0 or higher to ensure the graphical interface of the emulator renders correctly.

ii. Software Requirements

- **DevEco Studio:** The official Huawei IDE for HarmonyOS development.
- **HarmonyOS SDK:** Contains the necessary toolchains and libraries.
- **QEMU (v6.2+):** Used for simulating ARM/RISC-V architectures.
- **OpenHarmony Source/Images:** The open-source foundation of the OS.
- **Host OS:** Windows 10/11 or Ubuntu Linux (preferred for building source code).

d. Installation Steps (Step-by-Step with Figures)

Part 1: DevEco Studio Installation

The process began by installing DevEco Studio (v5.0.13). Unlike standard software, this IDE serves as the gateway to the entire HarmonyOS ecosystem, providing the necessary virtual device managers.

Figure 1: The successful launch of the DevEco Studio IDE environment.

Part 2: Attempting Virtual Device Creation

Within DevEco Studio, we accessed the Device Manager to configure a local emulator. This is intended to simulate a physical HarmonyOS device (Phone/Tablet/Watch) on the PC.

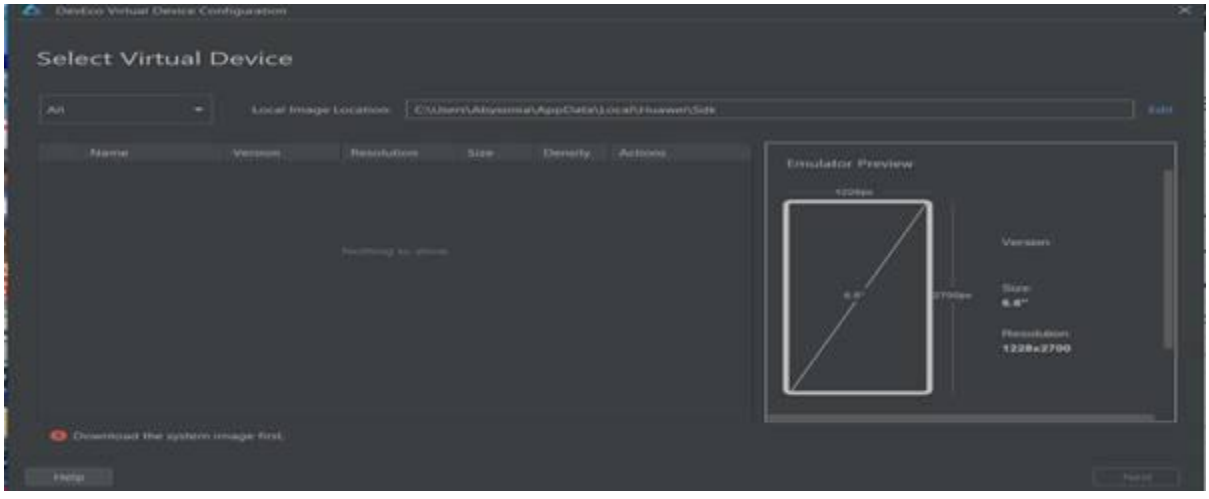

Figure 2: Emulator configuration interface with missing system image.

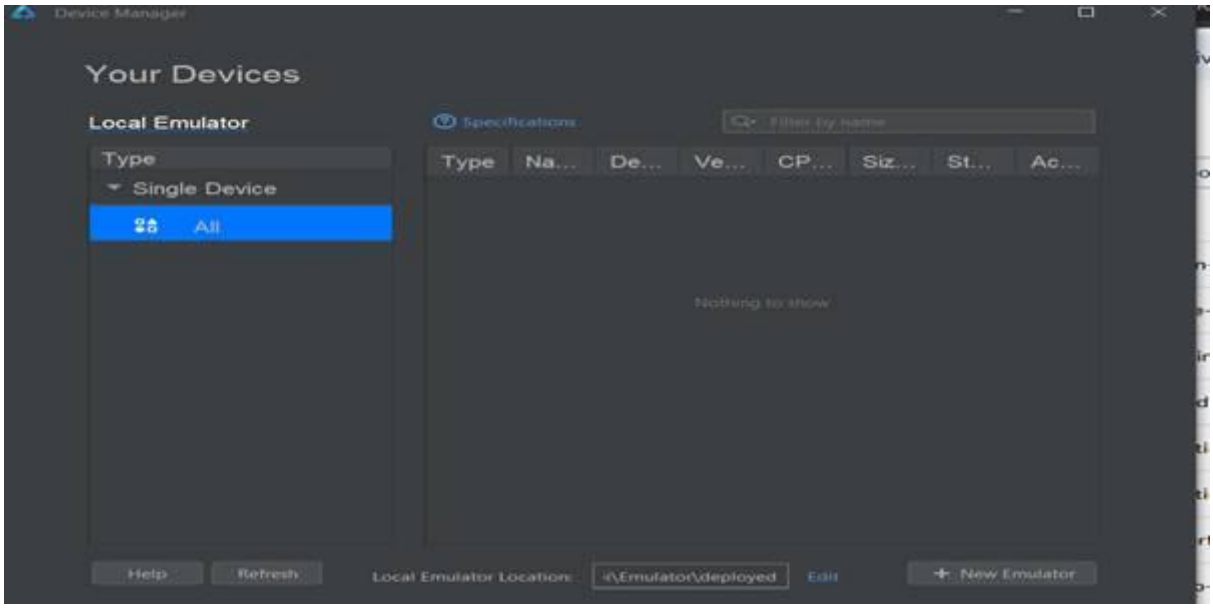

Figure 3: Device Manager showing no available emulators.

Part 3: Linux-Based OpenHarmony Setup (Terminal Configuration)

Due to limitations in the Windows-based IDE, an attempt was made to build the OpenHarmony environment on an Ubuntu host.

Step 1: Dependency Installation - The first step required installing the build dependencies, including Python 3, Git, and various compilers.

```
ubuntu@ubuntu: $ sudo apt install -y git python3 python3-pip python3-venv curl repo make g++ gcc gawk gperf unzip bison
flex patch libncurses5-dev libssl-dev liblz4-tool bc libxml2-utils qemu-system-arm gcc-arm-none-eabi

Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'libncurses-dev' instead of 'libncurses5-dev'
python3 is already the newest version (3.12.3-0ubuntu2).
python3 set to manually installed.
unzip is already the newest version (6.0-28ubuntu4.1).
unzip set to manually installed.
patch is already the newest version (2.7.6-7build3).
patch set to manually installed.
bc is already the newest version (1.07.1-3ubuntu4).
bc set to manually installed.
The following additional packages will be installed:
  binutils binutils-arm-none-eabi binutils-common
  binutils-x86-64-linux-gnu build-essential dpkg-dev fakeroot
  g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc-13
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu git-man ipxe-qemu
  ipxe-qemu-256k-compat-efi-roms javascript-common
  libalgorithm-diff-perl libalgorithm-diff-xs-perl
```

Figure 4: Terminal output displaying the installation of essential libraries and build tools.

Step 2: Repo Tool Setup - The repo tool is used to manage the dozens of Git repositories that make up the OpenHarmony project. We downloaded the tool and set the local path.

Figure 5: Downloading and configuring the repo tool to handle the source code manifest.

Step 3: Repository Initialization - We attempted to initialize the OpenHarmony 4.0 Release branch to begin the source code download.

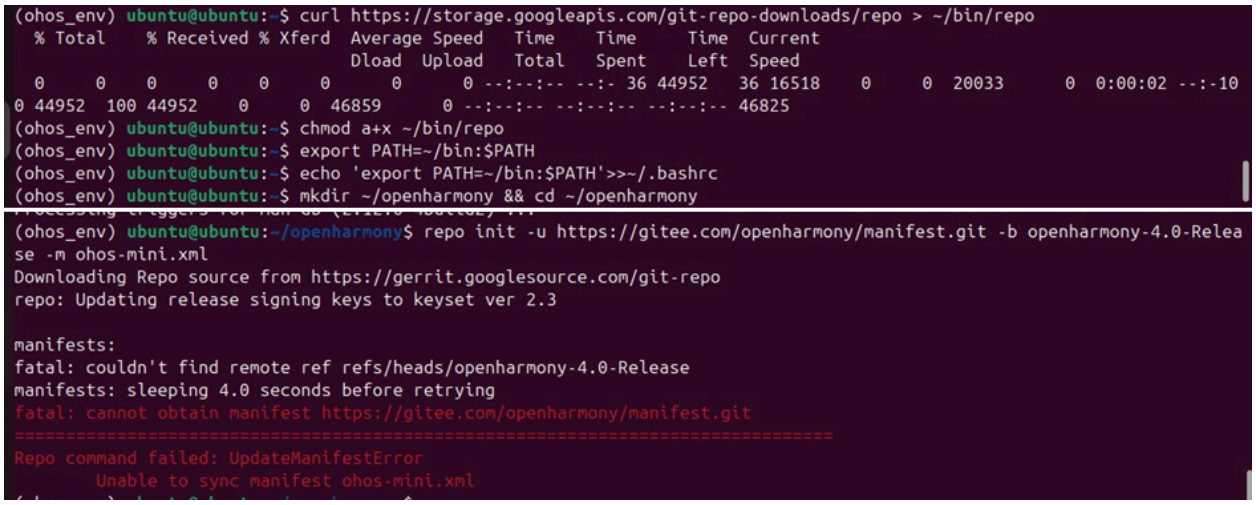

Figure 6: Attempting to synchronize the repository, resulting in a manifest error due to remote branch changes.

e. Issues (Problems Faced)

1. **System Image Block:** As shown in Figure 2, the emulator setup was halted by a "Download the system image first" error. These images were unavailable for direct download within the IDE.
2. **Manifest Failures:** During the repo init process, a "fatal: couldn't find remote ref" error occurred, indicating that the specific software branch was restricted or relocated.
3. **Storage Demands:** The source code size exceeds 31 GB, which creates a significant bottleneck for systems with limited bandwidth or disk space.
4. **Learning Curve:** The environment requires deep familiarity with GN and Ninja build systems, which are more complex than standard Windows installers.

f. Solutions Implemented

1. **IDE Sourcing:** Obtained a stable version of DevEco Studio from a third-party source when the official portal was inaccessible.
2. **Environment Isolation:** Used Python virtual environments (venv) to prevent dependency conflicts on the host system.
3. **Future Mitigation:** To solve the image issue, one would need a verified Huawei Developer Account to access the restricted download servers for system images.

g. Filesystem Support

HarmonyOS uses cutting-edge filesystems designed for flash memory and connectivity:

- **EROFS (Enhanced Read-Only File System):** A read-only system partition filesystem that provides high compression and superior read speeds.
- **HMDFS (Harmony Distributed File System):** This allows files to be shared across a network as if they were on a local drive—this is the core technology behind the "Super Device."
- **exFAT:** Recently added to support high-capacity external storage compatibility for users.

h. Advantages and Disadvantages

Advantages: Seamless cross-device integration, high security via microkernel isolation, and extreme efficiency on low-power IoT hardware.

Disadvantages: A significantly smaller app ecosystem compared to Android/iOS, and high difficulty of installation for non-enterprise developers.

i. Conclusion

HarmonyOS represents a major shift toward independent operating systems. While technically impressive, the installation process remains "closed" and difficult for beginners due to heavy reliance on specific system images and complex build tools.

j. Recommendation

Huawei should provide a standard Virtual Appliance (.OVA) for VirtualBox/VMware to allow students and researchers to test the OS without the 30GB+ overhead of building from source.

---

2. Virtualization in Modern Operating Systems

2.1 What is Virtualization?

Virtualization is a technology that uses software to create an abstraction layer over computer hardware. This allows the hardware elements of a single computer—processors, memory, storage, and more—to be divided into multiple virtual computers, commonly called Virtual Machines (VMs).

2.2 Why is Virtualization Used?

- **Efficiency:** It allows one physical server to do the work of many, reducing hardware waste.
- **Security:** VMs are isolated. If a virus infects a guest OS, the host system remains safe.
- **Legacy Support:** It allows modern computers to run old operating systems that are no longer supported by new hardware.
- **Cloud Computing:** Virtualization is the foundation of the cloud, allowing providers like AWS or Azure to serve millions of users on shared hardware.

2.3 How Virtualization Works

1. **Hypervisor:** The software (like VMware or VirtualBox) that creates and runs VMs.
2. **Shared Resources:** The hypervisor intercepts requests from the Guest OS and translates them for the physical hardware.
3. **Virtual Devices:** The VM "thinks" it has a real network card or hard drive, but it is actually a software-simulated device provided by the hypervisor.